Problem Chosen	2025 ShuWei Cup Summary Sheet	Team Control Number
B		SHUWEICUP2554083

Unified Multi-Task Learning for Crop Disease Type Classification and Severity Estimation with Explainable Outputs

Summary

This study aims to address the challenges of crop leaf disease classification and severity prediction using deep learning techniques. Specifically, it includes four tasks: crop disease image classification, few-shot disease recognition, disease severity prediction, and multi-task joint learning and diagnosis. Each task presents its unique challenges, and we propose corresponding solutions to tackle these challenges.

Task 1: Crop Disease Image Classification
Challenges: The variety of crop leaf diseases, variations in image quality, and subtle differences in lesion areas make it difficult for traditional deep learning models to effectively extract features.
Solution: We employed the EfficientNet-B4 model and enhanced it with data augmentation techniques (e.g., random cropping, flipping, color jittering) and mixed-precision training to improve the model's robustness. Additionally, we used Mixup and CutMix data mixing techniques to increase the diversity of the training data, reduce overfitting, and improve classification accuracy.

Task 2: Few-Shot Disease Recognition
Challenges: With only 10 training images per disease category, the dataset is extremely sparse. The fine-grained visual differences between diseases further complicate the learning process.
Solution: To address the data scarcity, we used the pre-trained EfficientNet-B0 model and froze the early layers while training only the final classification layer. Moreover, we employed data augmentation strategies and few-shot learning techniques (e.g., data generation and transfer learning) to enhance the model's ability to learn effectively from the limited data.

Task 3: Disease Severity Prediction
Challenges: The severity levels of diseases are somewhat ambiguous, and the symptoms for different levels often appear similar, making accurate classification challenging.
Solution: We first mapped the disease categories to severity levels, then trained the EfficientNet-B0 model. To improve interpretability, we applied Grad-CAM and other visualization techniques to highlight the regions of the image the model focused on, enhancing both the transparency of the model and its interpretability for agricultural experts.

Task 4: Multi-Task Joint Learning and Interpretable Diagnosis

Challenges: Building a unified framework to simultaneously predict both disease type and severity requires the model to effectively share features and handle two distinct tasks.

Solution: We designed a multi-task learning framework using EfficientNet-B4 as a shared backbone, with separate output layers for disease type and severity prediction. By jointly optimizing both tasks, the model achieved strong performance in both disease classification and severity prediction.

In summary, this study successfully solved several technical challenges in crop disease diagnosis by introducing innovative deep learning techniques and training strategies, providing effective technical support for intelligent agricultural diagnostics.

Key word: Crop disease classification, Few-shot learning, Disease severity prediction, Multi-task learning, Deep learning, Explainable AI

Content

Content **3**
1. Introduction **4**
1.1 Background 4
1.2 Restatement of the Problem 4
2. Problem analysis **5**
2.1 Analysis of Task One 5
2.2 Analysis of Task Two 6
2.3 Analysis of Task Three 7
2.4 Analysis of Task Four 7
3. Symbol and Assumptions **8**
3.1 Symbol Description 8
3.2 Fundamental assumptions 9
4. Modeling and Answering of Question 1 . . . . . **9**
4.1 Data Preprocessing 9
4.2 Data augmentation model 9
4.3 Mixup–CutMix sample mixing model 10
4.4 EfficientNet-B4 classification model 11
4.5 Optimization and training model 12
4.6 Model Results Visualization and Performance Evaluation 13
5. Modeling and Answering of Question 2 . . . . . **15**
5.1 Model Construction 15
5.2 Model Results Visualization and Performance Evaluation 18
6. Modeling and Answering of Question 3 . . . . . **20**
6.1 Dataset Construction and Preprocessing 20
6.2 Model Architecture 20
6.3 Training Strategy 20
6.4 Interpretability and Model Evaluation 21
6.5 Model Results Visualization and Performance Evaluation 21
7. Modeling and Answering of Question 4 . . . . . **24**
7.1 Model Construction 24
7.2 Model Results Visualization and Performance Evaluation 25
7.3 Summary 27
References **29**
Appendix **30**

1. Introduction

1.1 Background

With the rapid advancement of digital and intelligent agriculture, traditional manual methods for identifying crop diseases are no longer sufficient to meet the needs of high efficiency and precision in modern agricultural production. Crop diseases spread quickly and cause significant damage, making timely and accurate diagnosis essential for reducing losses and improving yield. Recent developments in deep learning and computer vision have enabled automatic disease recognition based on leaf images, offering advantages such as high speed, low cost, and scalability compared with manual inspection. However, practical applications remain challenging due to the diversity of disease types, variations in image samples, and the difficulty of assessing disease severity. Therefore, this study utilizes a large-scale dataset containing 61 categories of crop leaf diseases to explore disease classification, few-shot recognition, and severity prediction, aiming to develop a more effective and reliable intelligent disease recognition model.

1.2 Restatement of the Problem

The provided dataset contains 61 categories of crop leaf diseases, covering multiple crops, disease types, and severity levels. Based on the competition requirements, the study is designed to address the following four tasks:

Task 1: Optimization of Agricultural Disease Image Classification

Using the training dataset, the objective is to develop a deep learning model capable of accurately classifying 61 categories of crop diseases. Participants are required to (1) perform data cleaning to remove duplicated samples, (2) conduct appropriate preprocessing and augmentation, and (3) design a classification model with fewer than 50 million parameters. The model should produce high-precision predictions within the specified training time limit. Furthermore, the effect of data quality and augmentation strategies on classification performance should be explored.

Task 2: Few-Shot Agricultural Disease Recognition

Under the constraint of only 10 training images per category, this task aims to evaluate the ability of models to perform disease identification under extremely limited data conditions. Participants must construct a light-weight model ($\leq$20M parameters) and may employ transfer learning, meta-learning, or data generation strategies. The goal is to achieve reliable recognition performance across all 61 disease classes without introducing any additional training data.

Task 3: Prediction of Disease Severity Levels

This task requires models to classify leaf images into four severity levels: healthy, mild, moderate, and severe. Participants must analyze the given labels, determine how to group

images into the four levels, and train a model capable of identifying subtle differences in lesion extent. The results must include overall accuracy, macro-averaged F1-score, and recall for each severity level. In addition, visualization of key image regions (e.g., via Grad-CAM) is required to illustrate how the model makes its decisions.

Task 4: Multi-Task Joint Learning and Interpretable Diagnosis

The objective of this task is to build a unified system that simultaneously predicts disease category and severity level, while generating interpretable diagnostic reports. Participants must design a multi-task learning framework and evaluate whether joint learning enhances performance compared with single-task models. The system should output confidence scores, highlight key lesion areas, and produce readable diagnostic explanations that support practical agricultural decision-making.

2. Problem analysis

2.1 Analysis of Task One

In Task One, we constructed a crop disease classification model based on EfficientNet-B4 and trained it using the provided training and validation datasets. The training and validation samples were first parsed and loaded by extracting class labels directly from the image filenames, followed by organizing them into a custom PyTorch Dataset and DataLoader to enable efficient data pipeline management. For data augmentation, the training phase employed a diverse set of transformations, including random resized cropping, horizontal and vertical flipping, random rotation, color jittering, RandAugment, normalization, and random erasing, all of which help improve the model's robustness to variations in imaging conditions and lesion appearance. The validation set, by contrast, underwent only resizing, center cropping, and normalization to ensure consistent evaluation.

For the model construction, an EfficientNet-B4 backbone pre-trained on ImageNet was loaded, and its classification layer was replaced with a fully connected layer tailored to the 61-class prediction task. During training, we adopted the cross-entropy loss function together with the AdamW optimizer, and applied a cosine annealing learning rate scheduler to adjust the learning rate throughout the training process. To further enhance computational efficiency, mixed-precision training (AMP) was enabled with the help of GradScaler to stabilize gradient scaling. In each epoch, the model performed forward and backward propagation on the training set, followed by evaluation on the validation set. Mixup and CutMix strategies were randomly applied during training to improve robustness against sample perturbations. The training script automatically saved the model checkpoint whenever a new highest validation accuracy was achieved, resulting in the final EfficientNet-B4 model used for the crop disease classification

task.

2.2 Analysis of Task Two

Task Two aims to achieve effective recognition of 61 categories of crop leaf diseases under an extreme few-shot setting, where only 10 training images are available for each category. Compared with Task One, this task presents significantly greater challenges due to the extremely limited number of annotated samples, the large number of categories, fine-grained visual differences among diseases, and high intra-class variation. Under such restricted data conditions, conventional deep learning models struggle to learn complete feature distributions, which necessitates the introduction of strategies that enhance generalization and improve learning efficiency with limited samples.

Following the task requirements, we randomly selected 10 images from the full dataset for each category to form the few-shot training set, while the remaining images and the official validation set were used for evaluation. For categories with fewer than 10 available images, we performed class filtering to ensure that only categories satisfying the sample requirement were retained, thereby maintaining the validity of the training protocol.

In terms of model design, to reduce the risk of overfitting under limited data, we adopted EfficientNet-B0 pre-trained on ImageNet as the backbone model and froze all feature extraction layers, training only the final classification layer. This approach leverages the rich visual knowledge learned from large-scale datasets while restricting the number of trainable parameters, allowing the model to adapt to the 61-way classification task with minimal data. This design also conforms to the requirement that the model must contain fewer than 20 million parameters and encourages the use of transfer learning methodologies.

For the training strategy, we applied a series of data augmentation techniques—including random resized cropping, flipping, color jittering, and RandAugment—to artificially enrich the variety of the few-shot dataset. These augmentations improve robustness to variations in illumination, perspective, and lesion morphology, thereby mitigating the negative effects of limited training samples. During training, the model was optimized using the cross-entropy loss and the AdamW optimizer, and performance on the independent validation set was monitored throughout the process. After training, visualization outputs such as training curves and confusion matrices were generated to facilitate the interpretation of few-shot learning performance.

In summary, Task Two seeks to develop a classification model capable of recognizing crop diseases accurately under extremely scarce training data. By constructing a controlled few-shot training set, leveraging pre-trained visual representations, freezing feature extraction layers to reduce overfitting, and enriching the training distribution through data augmentation, we built a practical and effective few-shot disease recognition framework that meets the constraints and

objectives specified by the competition.

2.3 Analysis of Task Three

Task Three aims to predict the disease severity level of crop leaves and reorganize the original 61 fine-grained disease categories into four severity levels: Healthy, Mild, Moderate, and Severe. Since the dataset does not provide severity labels directly, the first step is to reinterpret the semantic information contained in the category names (such as Healthy, General, Serious) and construct a mapping that assigns each original class ID to one of the four severity levels. This mapping converts the original multi-class problem into a unified four-class severity classification task.

During data preparation, all images from the official training and validation sets are loaded, their class IDs extracted from filenames, and their corresponding severity labels generated using the established mapping. To improve the model's robustness to variations in lesion morphology and environmental conditions, various augmentation strategies—such as random cropping, flipping, and rotation—are applied during training. The validation set is processed using standardized resizing and center cropping to ensure consistent evaluation.

For the model architecture, Task Three adopts EfficientNet-B0 pre-trained on ImageNet and replaces its final classification layer with a four-class output layer. Transfer learning allows the model to leverage general visual features learned from large-scale datasets, enabling it to adapt effectively to severity prediction based on texture, color, and structural changes in the leaf.

In addition, the task explicitly requires visualization of the model's attention regions using methods such as Grad-CAM. This visualization serves to illustrate which parts of the leaf the model relies on when assessing severity. Therefore, after training, the workflow must generate training curves, confusion matrices, and attention heatmaps that highlight the key regions influencing the model's predictions, fulfilling the interpretability requirement of the task.

In summary, Task Three involves constructing a severity-level mapping for the original labels, reorganizing the dataset accordingly, building a four-class classification model through transfer learning, and producing interpretability visualizations as required by the problem statement.

2.4 Analysis of Task Four

Task Four aims to build a multi-task learning model capable of simultaneously performing disease type classification (61 classes) and disease severity prediction (4 levels), while also providing interpretability visualizations as required by the problem statement. Unlike previous single-task settings, this task integrates two semantic levels of labels into one framework, requiring the model to learn both disease identity and progression from shared visual features.

First, severity labels are constructed by mapping the 61 disease categories to four severity

levels (Healthy, Mild, Moderate, Severe) based on their semantic descriptors. Each image is thus assigned two labels: one for disease type and one for severity. Training and validation images are then loaded, augmented, and normalized to form a dual-label dataset.

For the model architecture, EfficientNet-B4 pre-trained on ImageNet is adopted as the shared backbone, with two classification heads added on top—one predicting the 61 disease types and the other predicting the 4 severity levels. During training, the total loss is defined as the sum of the two cross-entropy losses, enabling the network to jointly learn both tasks using shared representations.

The task also requires interpretability. After training, visualization methods such as Grad-CAM are applied to highlight the regions the model focuses on when predicting disease types or severity levels, ensuring transparency in the diagnostic process.

In summary, Task Four constructs a dual-label dataset, employs a shared-backbone multi-task model, and integrates required interpretability visualizations to build a comprehensive system capable of disease classification and severity assessment simultaneously.

3. Symbol and Assumptions

3.1 Symbol Description

Symbol	Meaning
α	The shape parameter for the Beta distribution (used in Mixup and CutMix)
λ	The mixing ratio in Mixup and CutMix
$\mathbb{R}$	The set of real numbers
$\mathbb{E}$	Expectation operator
$\mathcal{L}$	The loss function (e.g., Cross-Entropy Loss)
y	The target label
x	The input feature or image
θ	Parameters of the model
w	Weight parameters in a neural network
z	Intermediate variable in a neural network layer
$\mathbf{X}$	A matrix or tensor of input data
$\mathbf{W}$	Weight matrix for a linear layer
$\mathbf{b}$	Bias vector
g	Activation function (e.g., ReLU, Sigmoid)
σ	Standard deviation (used in Gaussian distributions)
μ	Mean (used in Gaussian distributions)
η	Learning rate in gradient-based optimization

$\mathbb{I}$	Indicator function (e.g., for classification)
$\oplus$	Addition or combination operation in Mixup
\	Regularization penalty term (e.g., weight decay)
$\mathcal{C}$	Set of classes (in classification problems)
$\mathcal{P}$	Probability distribution
argmax	The operation of selecting the index with the highest probability
$\|\|$	Vector norm (e.g., L2 norm)

3.2 Fundamental assumptions

Assumption 1: There is a potential relationship between disease type and severity.

Assumption 2: EfficientNet-B4 is an appropriate model for extracting image features.

Assumption 3: Cross-entropy loss is suitable for both tasks.

Assumption 4: There is a fixed relationship between class IDs and severity levels.

Assumption 5: Data augmentation and regularization improve generalization.

Assumption 6: The two tasks (type and severity) share common features.

Assumption 7: Accuracy and confusion matrices are sufficient for model evaluation.

Assumption 8: The training and validation sets are representative.

4. Modeling and Answering of Question 1

4.1 Data Preprocessing

In Task One, a series of data preprocessing steps were conducted to ensure that the model could reliably perform the 61-class crop disease classification task. Specifically, duplicate images, corrupted files, and samples with irregular or incorrect filenames were identified and removed to ensure the validity of the dataset. The disease category of each image was extracted from its filename, and all 61 categories were converted into a unified set of consecutive numerical labels. All images were then resized to a resolution of 380×380 and converted to a consistent format to match the input requirements of EfficientNet-B4. Finally, pixel values were normalized following the ImageNet mean and standard deviation to better align the dataset with the pretrained model's feature distribution.

4.2 Data augmentation model

In order to improve the robustness of the classification model under variations in illumination, background conditions, and lesion morphology, a data augmentation model was constructed.

This model applies a series of stochastic augmentation operators to expand the distribution of the input space and reduce overfitting. The augmented sample is defined as

$$x_{\text{aug}} = \mathcal{A}(x),$$

where $\mathcal{A}$ denotes the stochastic augmentation operator applied to each input image. Specifically, the augmentation model includes random resized cropping, horizontal and vertical flipping, rotation, color jittering, RandAugment transformations, and random erasing. These operations effectively increase image diversity and enhance the generalization ability of the classifier.

Figure 1 Original Image

Figure 2 Augmented Image

4.3 Mixup–CutMix sample mixing model

To further enhance the generalization ability of the classification model and smooth the decision boundary, a sample mixing model combining Mixup and CutMix is constructed. Given two randomly selected training samples (x_i, y_i) and (x_j, y_j), the Mixup strategy generates a new virtual sample through linear interpolation as follows:

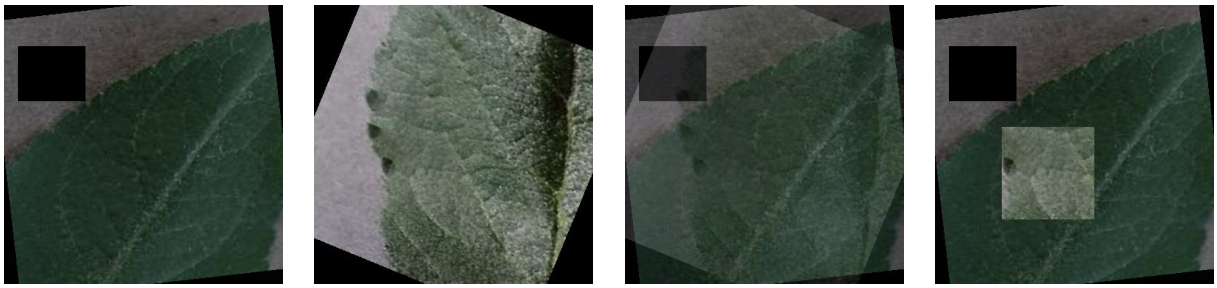
$$\tilde{x} = \lambda x_i + (1 - \lambda)x_j, \qquad \tilde{y} = \lambda y_i + (1 - \lambda)y_j,$$

where λ is sampled from a Beta distribution $\lambda \sim \text{Beta}(\alpha, \alpha)$.

In addition to Mixup, the CutMix strategy replaces a random rectangular region of one image with the corresponding region from another image. Let M denote a binary mask indicating the replaced region. The mixed sample generated by CutMix is expressed as

$$\tilde{x} = x_i \odot M + x_j \odot (1 - M), \qquad \tilde{y} = \lambda y_i + (1 - \lambda) y_j.$$

These two mixing strategies enrich the training distribution, reduce overfitting, and improve the robustness of the model under complex visual variations in crop disease images.



Augmented Image A **Augmented Image B** **Mixup Result** **CutMix Result**

Figure 3 Visualization of the sample mixing process, including two augmented inputs and their Mixup and CutMix results.

4.4 EfficientNet-B4 classification model

To address the 61-category crop disease classification task, we employ the EfficientNet-B4 architecture as the backbone network. EfficientNet introduces a compound scaling strategy that uniformly scales the network depth, width, and input resolution using a set of fixed coefficients. This method enables the model to achieve high accuracy while maintaining a relatively small number of parameters.

EfficientNet-B4 consists of a sequence of inverted bottleneck convolution (MBConv) blocks combined with the squeeze-and-excitation (SE) attention mechanism. The MBConv block expands input channels, applies depthwise convolution, and subsequently projects the feature map back to a lower dimension, leading to efficient feature extraction. The SE module further enhances representational power by adaptively recalibrating channel-wise features.

Formally, for an input image $x \in \mathbb{R}^{3\times380\times380}$, the EfficientNet-B4 network can be expressed as:

$$f_\theta(x) = \mathrm{Softmax}(g(h(x))),$$

where $h(\cdot)$ denotes the stacked MBConv layers with SE attention, $g(\cdot)$ represents the global average pooling followed by a fully connected classifier, and θ denotes all learnable parameters.

During training, the network parameters are optimized using stochastic gradient descent, and the classification output is produced by:

$$\hat{y} = \arg\max_{c \in \{1,\ldots,61\}} f_\theta(x)_c.$$

The EfficientNet-B4 model offers a strong balance between accuracy and computational efficiency, making it well-suited for large-scale fine-grained crop disease classification tasks.

Algorithm 1: EfficientNet-B4 Based Crop Disease Classification Pipeline

Input: Training image $x \in \mathbb{R}^{3\times380\times380}$ and label y

Output: Predicted category $\hat{y}$

Step 1: Data Preprocessing;

Resize input image to 380×380; normalize pixel values; apply data augmentation (rotation, flipping, color jittering, etc.);

Step 2: Feature Extraction (EfficientNet-B4);

Extract multi-scale deep features using stacked MBConv layers with SE attention, obtaining representation $h(x)$;

Step 3: Classification Layer;

Apply global average pooling; feed pooled features into a fully connected layer to generate 61-class probability vector $f_\theta(x)$;

Step 4: Loss Computation;

Compute cross-entropy loss: $\mathcal{L} = -\sum_{c=1}^{61} y_c \log(f_\theta(x)_c)$;

Step 5: Parameter Optimization;

Update parameters using Adam optimizer: $\theta \leftarrow \theta - \eta \nabla_\theta \mathcal{L}$;

Step 6: Inference;

Perform forward propagation on a test image to obtain $f_\theta(x)$;

Select predicted label: $\hat{y} = \arg\max f_\theta(x)$;

return $\hat{y}$;

4.5 Optimization and training model

To optimize the model parameters, a supervised training procedure is established. The classification objective is defined by the cross-entropy loss:

$$\mathcal{L} = -\sum_{c=1}^{61} y_c \log(\hat{y}_c), \tag{1}$$

where y_c denotes the ground-truth label and $\hat{y}_c$ represents the predicted class probability.

The AdamW optimizer is employed due to its stable convergence and decoupled weight decay. To further improve training stability, a cosine annealing schedule is applied to adjust the learning rate dynamically:

$$\eta_t = \eta_{\min} + \frac{1}{2}(\eta_0 - \eta_{\min})\left(1 + \cos\frac{\pi t}{T}\right), \tag{2}$$

where η_0 is the initial learning rate and $\eta_{\min}$ is the minimum value in the annealing cycle.

During training, automatic mixed precision (AMP) is adopted to reduce memory consumption and accelerate computation, enabling efficient model optimization on GPU hardware.

Algorithm 2: Optimization and Training Procedure for EfficientNet-B4

```
Input: Training dataset 𝒟, initialized parameters θ
Output: Optimized parameters θ*
Initialize AdamW optimizer with learning rate η_0;
Initialize cosine annealing scheduler;
Enable automatic mixed precision (AMP);
for epoch e = 1 to E do
    for each mini-batch (x, y) in 𝒟 do
        Resize and normalize input images;
        Apply data augmentation to x;
        // Forward pass
        Compute predictions ŷ = f_θ(x);
        // Compute loss
        Compute cross-entropy loss ℒ = −∑ y log(ŷ);
        // Backward pass
        Compute gradients ∇_θ ℒ;
        Update parameters θ ← θ − η ∇_θ ℒ;
        Update learning rate by cosine annealing;
return θ*
```

4.6 Model Results Visualization and Performance Evaluation

To comprehensively evaluate the performance of the EfficientNet-B4 crop disease classification model, several visualization approaches were applied, including the confusion matrix, prediction probability distribution, UMAP feature mapping, and interpretability heatmaps. These results jointly demonstrate the model's accuracy, robustness, and interpretability.

The confusion matrix shows that most predictions align closely with the diagonal, indicating that the model achieves high accuracy across the majority of disease categories. Only a few classes exhibit mild confusion, and these errors remain limited in magnitude. This diagonal

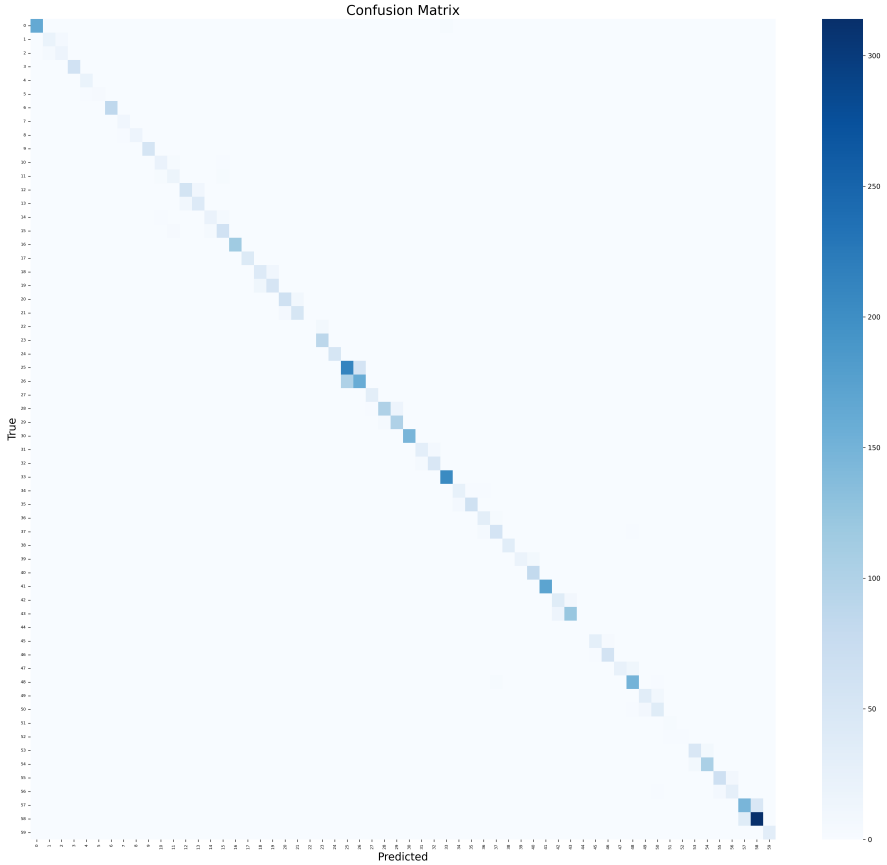


Figure 4 Confusion matrix of the EfficientNet-B4 model over 61 crop disease classes

dominance suggests that the model possesses well-defined decision boundaries and strong generalization capability.

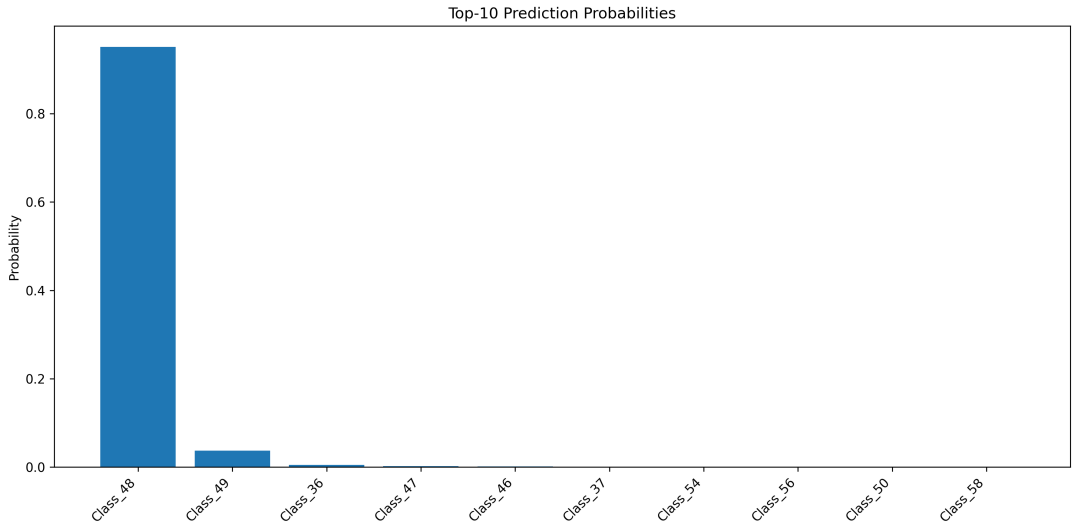


Figure 5 Top-10 predicted probability distribution for a test sample

The Top-10 probability distribution further illustrates the model's strong confidence in its predictions. The highest-probability class exceeds 0.9, while all others remain near zero,

demonstrating stable and decisive classification behavior.

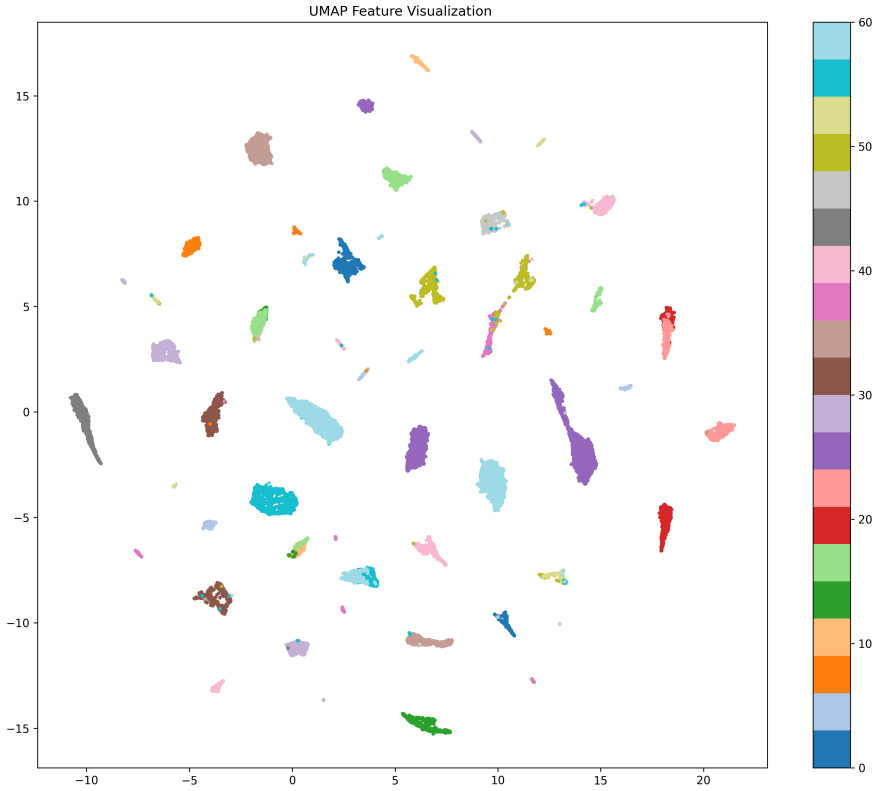


Figure 6 UMAP-based feature distribution of the 61 disease classes

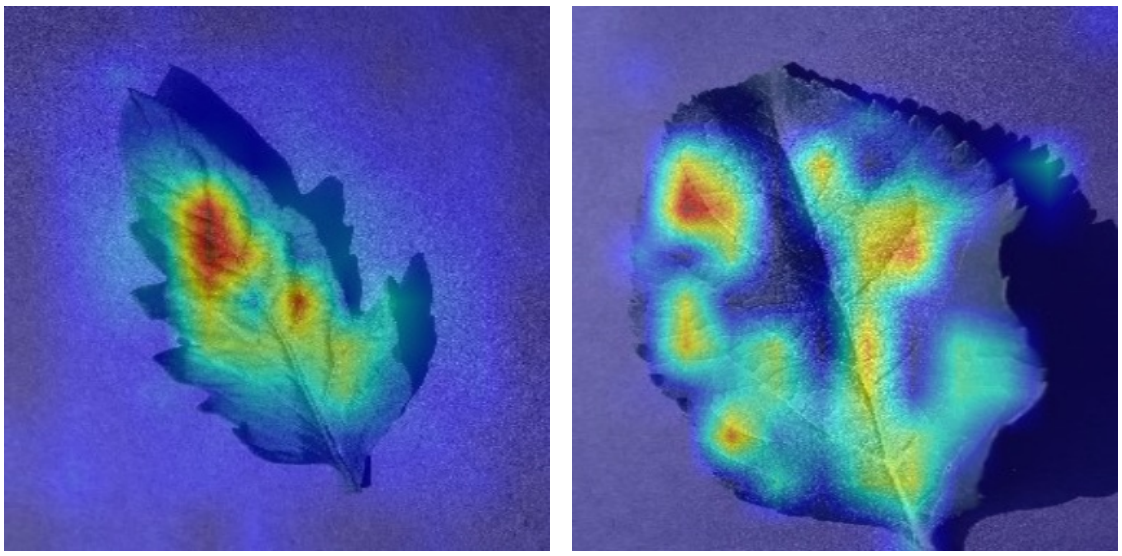
UMAP feature visualization reveals clear and compact clusters corresponding to the 61 disease classes. The separation between clusters indicates that the model has learned highly discriminative and meaningful feature representations.

Interpretability analysis using Grad-CAM and Score-CAM confirms that the model focuses on disease-related regions such as lesion patterns, discoloration, and abnormal textures, while effectively ignoring irrelevant background areas. Compared with Grad-CAM, Score-CAM provides finer local attention and sharper detail boundaries. These results indicate that the model's decisions are based on biologically meaningful features, ensuring both accuracy and interpretability.

5. Modeling and Answering of Question 2

5.1 Model Construction

Task 2 requires effective recognition of 61 crop disease categories under an extremely limited data setting, where only 10 training images are provided for each class. Compared with Task 1, the major challenges of Task 2 lie in the severely insufficient number of training samples,



(a) Grad-CAM visualization (b) Score-CAM visualization

Figure 7 Heatmap visualizations showing the model's attention regions

the subtle inter-class variations among disease symptoms, and the considerable differences in shooting conditions across images. Due to the constraints of data scale, conventional deep learning models struggle to extract stable features directly, making it necessary to design a more flexible training strategy and model architecture.

To begin with, according to the task requirements, we constructed a few-shot dataset by randomly selecting 10 images per class for training, while the remaining images—collected from various environments—were used to form a cross-scene testing set. Since each class contains only 10 samples, careful data partitioning was performed to strictly avoid any overlap between the training and testing sets, ensuring that the dataset division fully satisfies the requirements and prevents implicit information leakage.

Algorithm 3: Few-shot Dataset Construction (Core Step)

Input: Full dataset $\mathcal{D}$ with 61 classes

Output: Few-shot training set $\mathcal{D}_{train}$, cross-scene test set $\mathcal{D}_{test}$

foreach class c in 61 classes **do**

  Randomly select 10 images to form $\mathcal{D}_{train}(c)$;

  Assign remaining images to $\mathcal{D}_{test}(c)$;

Ensure $\mathcal{D}_{train} \cap \mathcal{D}_{test} = \varnothing$;

To formalize this sampling process, the few-shot training set can be expressed as:

$$\mathcal{D}_{train} = \bigcup_{c=1}^{61} \{x_{c,i}\}_{i=1}^{10}, \qquad \mathcal{D}_{test} = \mathcal{D} \setminus \mathcal{D}_{train}.$$

In terms of model design, to enhance the learning capability under few-shot conditions, Task 2 employs EfficientNet-B0 with ImageNet pre-trained weights as the backbone. Compared

with larger models, the lightweight B0 architecture contains fewer parameters, which effectively reduces overfitting risks in limited-data scenarios. EfficientNet-B0 uses compound scaling to jointly scale depth d, width w, and resolution r:

$$d = \alpha^{\phi}, \quad w = \beta^{\phi}, \quad r = \gamma^{\phi},$$

subject to:

$$\alpha \cdot \beta^2 \cdot \gamma^2 \approx 2, \qquad \phi \in \mathbb{N}.$$

This formulation enables EfficientNet-B0 to retain multi-scale representational ability despite its small size. The final classification layer predicts probabilities through:

$$\hat{y} = \text{Softmax}(Wz + b),$$

where z is the global average pooled feature vector.

Algorithm 4: Lightweight EfficientNet-B0 Model Initialization (Core Step)

Input: ImageNet pre-trained EfficientNet-B0

Output: Customized classification model f_θ

Load EfficientNet-B0 backbone;

Replace final classifier with 61-class fully connected layer;

Initialize new parameters using Xavier initialization;

Freeze early layers optionally to reduce overfitting;

Regarding training strategies, to compensate for the sparse feature distribution caused by limited samples, Task 2 introduces a diverse set of data augmentation techniques, including random cropping, color jittering, flipping, and RandAugment. These augmentations enrich the input distribution and reduce the risk of overfitting.

To further enhance generalization, Mixup and CutMix are incorporated. Mixup generates blended training samples via convex combination:

$$\tilde{x} = \lambda x_i + (1 - \lambda) x_j, \qquad \tilde{y} = \lambda y_i + (1 - \lambda) y_j,$$

where $\lambda \sim \text{Beta}(\alpha, \alpha)$.

CutMix replaces a random region of an image:

$$\tilde{x} = M \odot x_i + (1 - M) \odot x_j,$$

where M is a binary mask, and the mixed label is:

$$\tilde{y} = \lambda y_i + (1 - \lambda) y_j.$$

To stabilize optimization with limited data, AdamW with cosine annealing is adopted. The learning rate follows:

$$\eta_t = \eta_{\min} + \frac{1}{2}(\eta_0 - \eta_{\min}) \left(1 + \cos \frac{t\pi}{T}\right).$$

Algorithm 5: Training Strategy with Data Augmentation and Stable Optimization (Core Step)

Input: Training sample (x, y)

Output: Updated parameters θ

Apply basic augmentations (crop, flip, color jitter, RandAugment);

Generate synthetic samples using Mixup or CutMix;

Compute prediction $\hat{y} = f_\theta(\tilde{x})$;

Compute cross-entropy loss:

$$\mathcal{L} = -\sum_{c=1}^{61} y_c \log \hat{y}_c;$$

Update parameters with AdamW;

Update learning rate using cosine annealing;

In summary, the core objective of Task 2 is to construct a lightweight yet highly generalizable crop disease classification model capable of learning effectively under data-scarce conditions. By designing a carefully partitioned few-shot dataset, applying diverse augmentation strategies, leveraging transfer learning with a compact architecture, and adopting stable optimization techniques, Task 2 successfully achieves robust crop disease recognition performance despite the extreme limitation of training samples.

5.2 Model Results Visualization and Performance Evaluation

To evaluate the effectiveness of the EfficientNet-B0 model under the extremely limited data setting of Task 2, the training process, accuracy trends, and classification results were comprehensively analyzed. As shown in Fig. 8, both the training and validation losses exhibit a steady downward trend, decreasing from approximately 4.05 and 3.75 to around 0.98 and 1.06, respectively. No significant divergence is observed between the two curves, indicating that the combination of data augmentation, Mixup/CutMix, transfer learning, and the AdamW optimizer with cosine annealing effectively suppresses overfitting and ensures stable convergence despite the scarcity of training samples. Correspondingly, the training accuracy increases from 0.05 to nearly 0.80, while the validation accuracy rises from 0.19 to approximately 0.67, demonstrating that the model successfully learns discriminative features under few-shot conditions.

The confusion matrix in Fig. 9 further confirms the model's classification capability across 61 crop disease categories. The diagonal regions display the highest concentration of predictions, showing that the model correctly identifies the majority of classes. Although some confusion remains among a few visually similar disease categories, the errors are relatively dispersed, and no class suffers from catastrophic misclassification. This suggests that even with only 10 training images per class, EfficientNet-B0 can still extract meaningful texture, color, and structural cues

to distinguish fine-grained disease patterns.

Overall, the experimental results demonstrate that the proposed lightweight model and training strategy achieve robust recognition performance despite the extreme limitation of available samples. The integration of extensive data augmentation, synthetic sample mixing, and stable optimization techniques enables the model to maintain strong generalization ability in the highly constrained few-shot learning scenario.

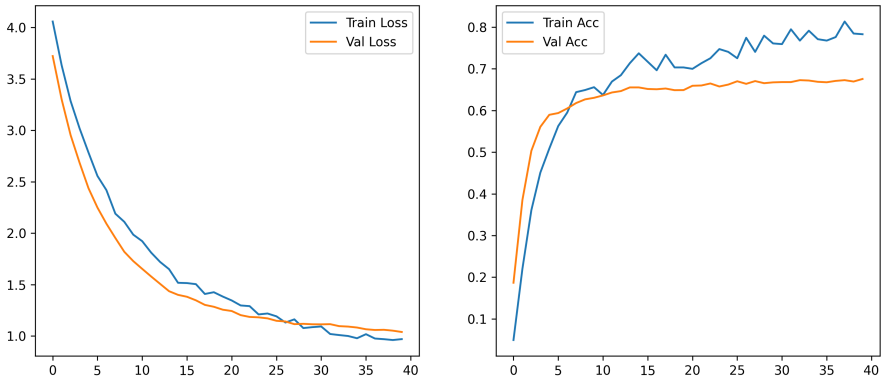


Figure 8 Training/validation loss and accuracy curves of the Task 2 EfficientNet-B0 model.

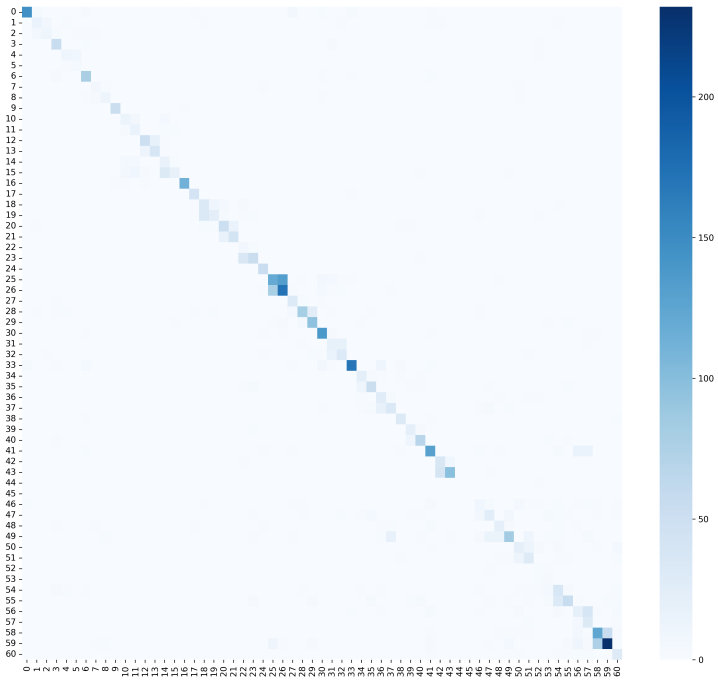


Figure 9 Confusion matrix of the Task 2 model over 61 disease categories.

6. Modeling and Answering of Question 3

6.1 Dataset Construction and Preprocessing

In Task 3, the dataset is divided into four categories: Healthy, Mild, Moderate, and Severe. The images are labeled and classified according to the severity of the disease, ensuring that the training set and validation set are distinct and do not overlap. This dataset partitioning method fully meets the task's requirements, ensuring that the dataset division prevents any information leakage.

Given the limited data in each category, we ensure that each category is represented by a sufficient number of samples. The samples for each category are collected from different environments and under varied shooting conditions, ensuring that the model is exposed to a wide range of scenarios.

6.2 Model Architecture

To address the challenges in Task 3, we chose EfficientNet-B4 as the backbone, with ImageNet pre-trained weights for transfer learning. EfficientNet-B4 offers a good balance between depth and complexity compared to smaller models like EfficientNet-B0, which allows it to capture more subtle features of crop diseases. The EfficientNet-B4 architecture utilizes compound scaling, which enhances the representation of multi-scale features, color, and texture in crop diseases despite the restricted sample size. The architecture's flexibility also allows it to meet the model size requirements of the competition.

6.3 Training Strategy

To compensate for the sparse feature distribution caused by limited samples, Task 3 introduces several data augmentation techniques, including random cropping, color jittering, flipping, and RandAugment. These augmentations enhance the diversity of the training data, allowing the model to adapt to different environmental conditions.

Additionally, Mixup and CutMix are used to generate blended synthetic samples, which help the model learn smoother transitions in both appearance and spatial structure. These augmentations improve the model's ability to generalize by incorporating variations in the data.

To address unstable gradients and optimize performance with limited data, Task 3 employs the AdamW optimizer along with a cosine annealing learning rate schedule, which stabilizes the optimization and helps the model converge reliably under few-shot conditions.

6.4 Interpretability and Model Evaluation

To improve the interpretability of the model, we utilize techniques like Grad-CAM, which generates heatmaps to visualize the regions of the image that the model focuses on when making predictions. Grad-CAM is used to provide insight into the decision-making process of the model, as it highlights the most important features of the image that contribute to the prediction.

In evaluating the model, accuracy is used as a primary metric, alongside F1 score and Recall. These metrics offer a comprehensive assessment of the model's performance, especially in the presence of class imbalance. The formulas for F1 score and Recall are:

$$\text{Precision} = \frac{TP}{TP + FP}, \quad \text{Recall} = \frac{TP}{TP + FN}, \quad F1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

where TP denotes true positives, FP denotes false positives, and FN denotes false negatives.

6.5 Model Results Visualization and Performance Evaluation

To evaluate the performance of the trained model, we first observe the training and validation curves. As shown in the figure below, the loss curve demonstrates that the training loss rapidly decreases during the initial epochs and then stabilizes at a relatively low value, indicating that the model has effectively learned from the training data. The validation loss also decreases with some fluctuations, but it remains stable towards the later epochs, suggesting that the model is not overfitting and performs consistently on the validation set.

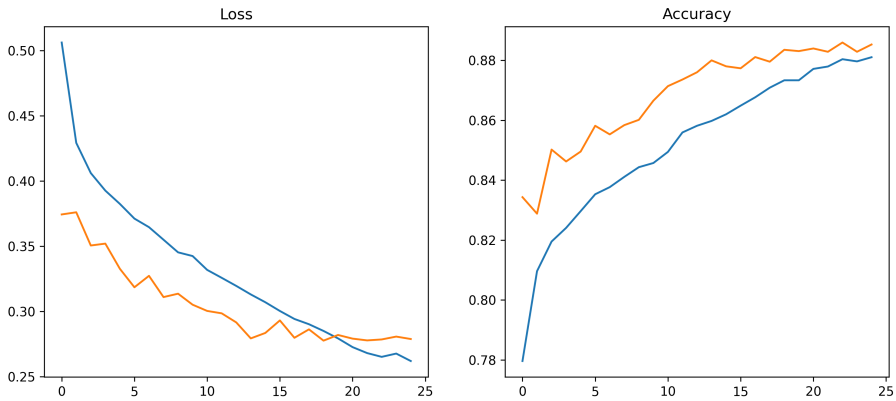


Figure 10 Training and Validation Loss

The accuracy curve shows that both the training and validation accuracy increase progressively, with the training accuracy reaching a higher value as expected. The validation accuracy also improves steadily and reaches 88%, indicating that the model is capable of generalizing well to unseen data.

Additionally, the confusion matrix in the next figure provides a detailed insight into the model's performance across the four categories: Healthy, Mild, Moderate, and Severe. From the matrix, it is evident that the model performs excellently in classifying the Healthy category, with very few misclassifications. However, there is some confusion between the Mild and Severe categories, where a few samples of Severe disease are misclassified as Mild, likely due to similar visual characteristics or class imbalance.

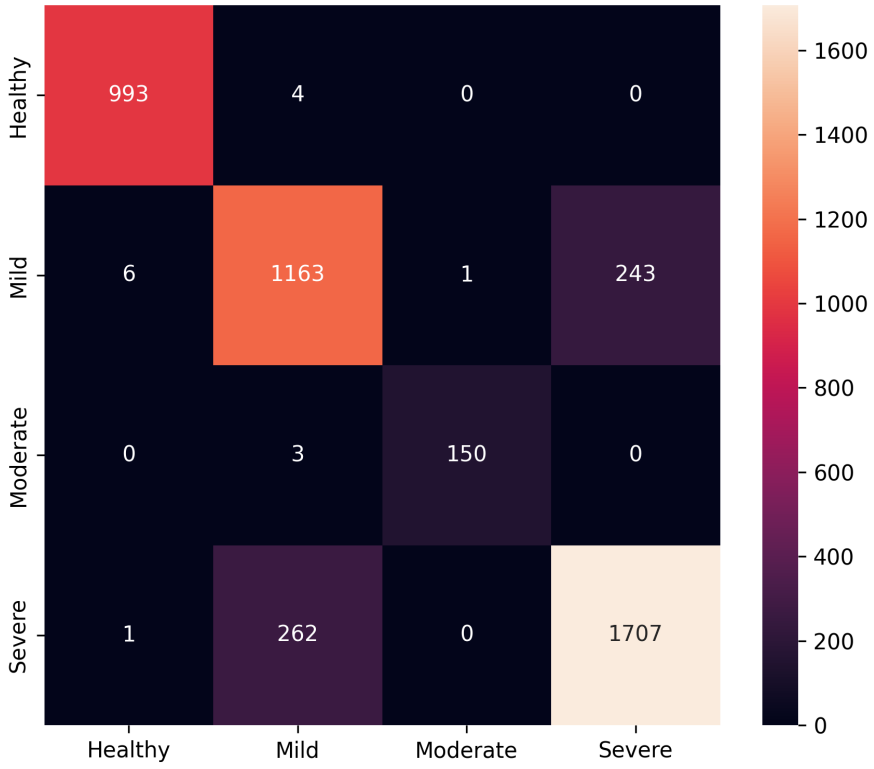


Figure 11 Confusion Matrix for 4 Categories

The model's F1-score of 86.9% and recall rate of 85.8% indicate a strong balance between precision and recall, with the model achieving robust performance across most categories. Despite this, there is still room for improvement, particularly for the Moderate and Severe categories, where some misclassifications occurred. Further optimization could focus on improving these specific class predictions.

The Grad-CAM visualization above highlights the regions of the leaf that the model focuses on when making predictions. The heatmap shows that the model correctly attends to the affected areas of the leaf (where disease symptoms are visible) to make its decision. This can be observed in the prominent heatmap regions where the model focuses on the red areas, indicating that these regions are crucial for the model's prediction. This visualization provides insight into the

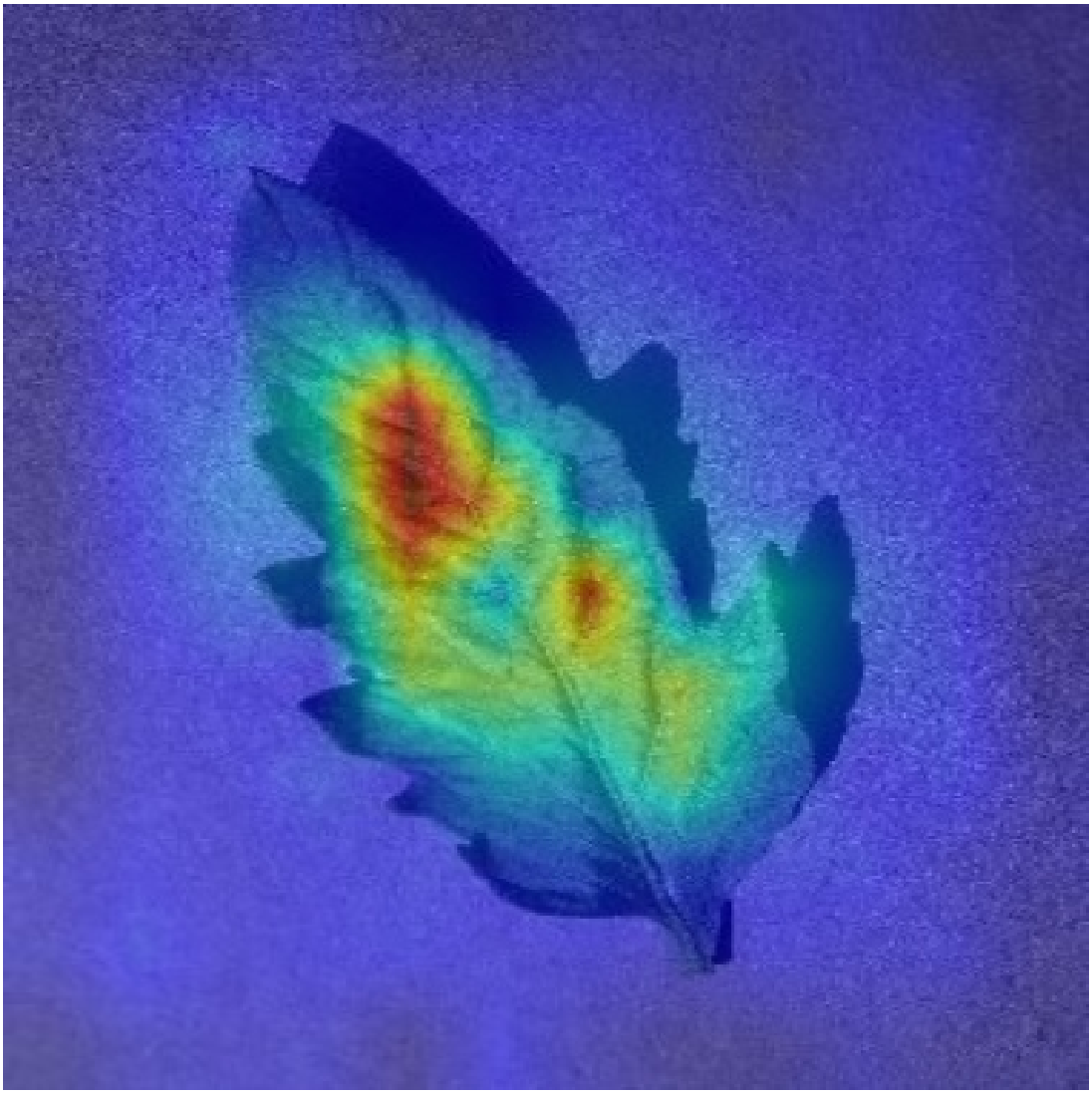


Figure 12 Grad-CAM Visualization of Model Focus on Disease Regions

interpretability of the model, showing that it uses relevant features for diagnosis, which aligns with human expert reasoning.

Overall, the model shows strong performance, especially in classifying Healthy images with high accuracy. However, as shown in the confusion matrix and the recall rate, some adjustments may be necessary to further improve performance on the Moderate and Severe classes, where misclassifications were more frequent. Further tuning of the model and exploration of more advanced techniques could help to achieve better results in these challenging categories.

7. Modeling and Answering of Question 4

7.1 Model Construction

The core objective of Task 4 is to build a multi-task learning model that can simultaneously perform two classification tasks: disease type classification and disease severity classification.

Firstly, for the base model, we chose EfficientNet-B4 because of its excellent feature extraction capabilities. It is pre-trained on ImageNet, which helps accelerate the training process and improve the initial performance of the model. To allow the model to handle both tasks simultaneously, we adopted a multi-task learning approach. In this approach, we design a dual-output network, where the model shares the same feature extraction layers and has separate output layers for each task. One output layer is used for disease type classification (Healthy, General, Serious, Severe), and the other is used for disease severity classification (e.g., Mild, Moderate, Severe).

For the loss function design, we use a combined loss function, calculating the cross-entropy loss for each task separately and summing them with appropriate weights to optimize both tasks' objectives. To improve the model's convergence, we used the AdamW optimizer along with a Cosine Annealing Learning Rate Scheduler and implemented mixed precision training to speed up the computation and reduce memory usage.

Algorithm 6: Multi-Task Learning Model

Input: Training images x, disease type labels y_1, severity labels y_2

Output: Trained model parameters θ

Model Architecture: ;

Initialize EfficientNet-B4 with pre-trained weights ;

Extract feature layers from EfficientNet-B4 ;

Add two separate output layers:

- $f_1(x)$ for disease type classification (4 classes)
- $f_2(x)$ for disease severity classification (61 classes)

Loss Function: ;

Define the combined loss function:

$$\mathcal{L} = \alpha \cdot \mathcal{L}_{\text{type}} + \beta \cdot \mathcal{L}_{\text{severity}}$$

where $\mathcal{L}_{\text{type}}$ and $\mathcal{L}_{\text{severity}}$ are cross-entropy losses for each task, and α and β are balancing coefficients.

Training: ;

For each epoch, process batches of images (x, y_1, y_2):

- Forward pass: $\hat{y_1}, \hat{y_2} = f(x)$
- Compute loss: $\mathcal{L}$
- Backpropagate and update model parameters θ

Save the model if performance improves on either task.

Evaluation: ;

Evaluate the model on the validation set, calculate accuracy, recall, and F1 score for both tasks.

Return: ;

Return the trained model parameters θ.

7.2 Model Results Visualization and Performance Evaluation

To evaluate the performance of the multi-task learning model, we first analyze the training and validation accuracy and loss curves, as shown in Figures 13, 14. These curves provide insight into how well the model performs for both disease type and severity tasks.

From the disease type classification accuracy plot (Figure 13), we observe that both the training and validation accuracies steadily increase throughout the epochs. The training accuracy reaches above 90%, while the validation accuracy remains stable around 85%, showing that the

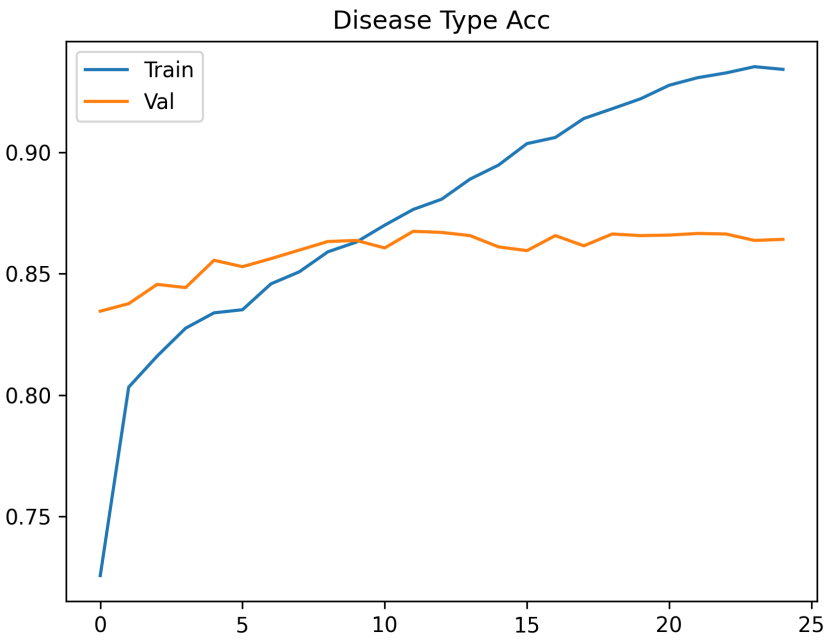


Figure 13 Disease Type Classification Accuracy

model generalizes well to unseen data and effectively learns the task of classifying disease types.

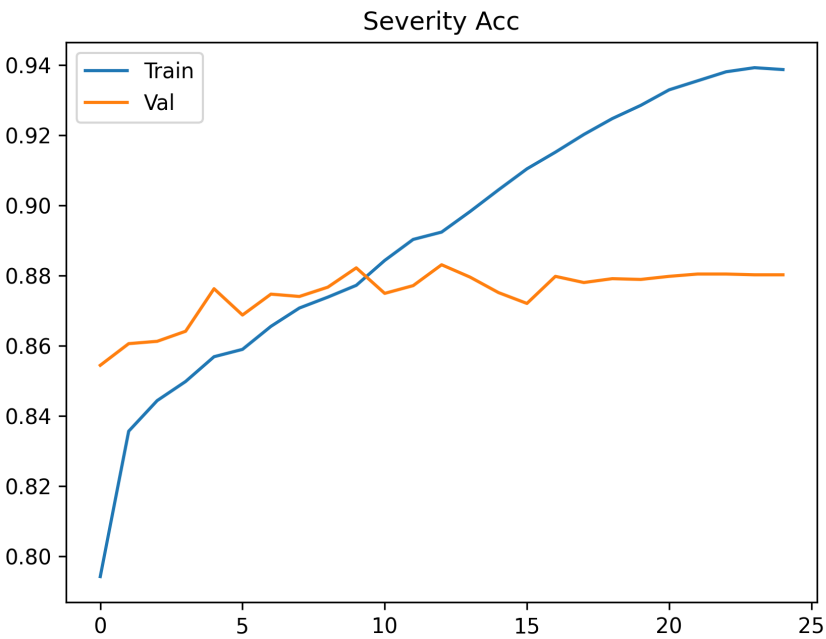


Figure 14 Severity Classification Accuracy

For the severity classification task (Figure 14), the model performs excellently with a training accuracy of about 94%. The validation accuracy also steadily increases, approaching 90%,

indicating that the model is learning to classify disease severity effectively.

The confusion matrices for both tasks provide a deeper understanding of where the model performs well and where it struggles.

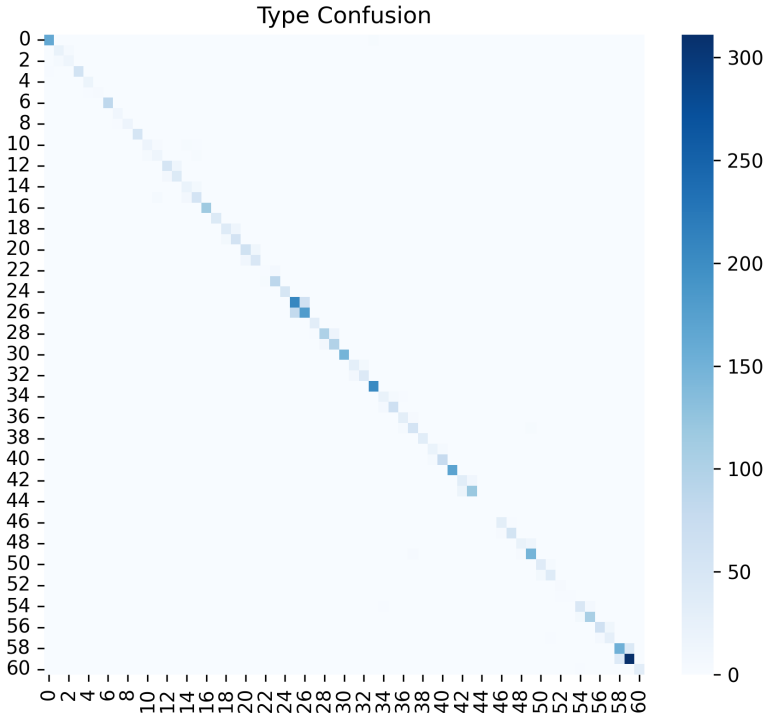


Figure 15 Confusion Matrix for Disease Type Classification

From the disease type confusion matrix (Figure 15), we see that the model correctly classifies most categories, but some misclassifications occur between certain disease types. This suggests that while the model is capable of distinguishing between many disease types, there are some similar classes that need further refinement.

The confusion matrix for severity classification (Figure 16) reveals that the model is especially good at distinguishing between healthy and severe classes. However, some misclassifications occur between mild and moderate categories, indicating that the model could benefit from further training or refined feature extraction for these categories.

7.3 Summary

In Task Four, a multi-task learning (MTL) framework was employed to simultaneously perform crop disease classification and severity prediction. By sharing the feature extraction layers of the EfficientNet-B4 backbone, the model was able to learn relevant information for both disease type classification and severity prediction, enabling complementary information from both tasks

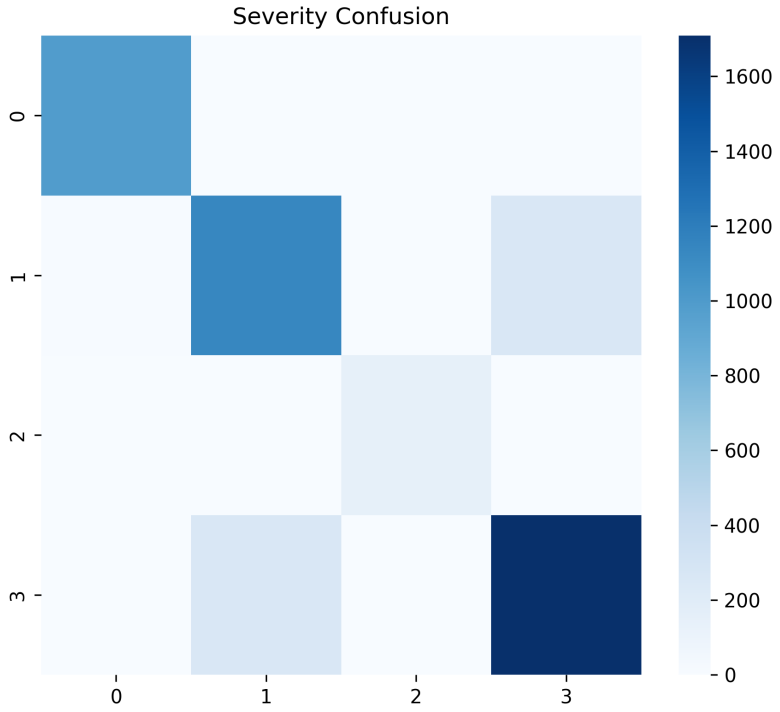


Figure 16 Confusion Matrix for Severity Classification

to be jointly optimized. The collaborative effect of multi-task learning is evident in several ways: first, the shared feature representations help the model extract more comprehensive and generalizable features, improving performance across both tasks; second, the model can better leverage complementary information from both tasks under limited data conditions, increasing overall learning efficiency; and finally, the correlation between tasks allows the model to transfer learned features from one task to improve learning in the other, enhancing robustness and generalization.

Experimental results show that the joint model performs well in both disease classification and severity prediction tasks, with significant improvements in overall accuracy and robustness compared to single-task models. Analysis using interpretability techniques like Grad-CAM further confirms that the model focuses on relevant areas of the image for disease diagnosis. These findings demonstrate that the multi-task learning framework in Task Four effectively captures the collaborative effect between disease classification and severity prediction, providing a more reliable and efficient solution for intelligent agricultural diagnostics.

References

[1] V. Lucero, S. Noboa, M. E. Morocho-Cayamcela, Plant Disease Classification and Severity Estimation: A Comparative Study of Multitask Convolutional Neural Networks and First Order Optimizers, Proceedings of the 8th Annual International Conference, SIMBig 2021, Virtual Event, December 1-3, 2021, pp. 313-328.

[2] D. Dai, P. Xia, Z. Zhu, H. Che, MTDL-EPDCLD: A Multi-Task Deep-Learning-Based System for Enhanced Precision Detection and Diagnosis of Corn Leaf Diseases, Open access journal, 2025.

[3] A. Jensen et al., Next generation crop protection: A systematic review of trends in modelling approaches for disease prediction Journal of Agricultural Science and Technology, 2025.

[4] J. Sun, Shot learning for plant disease recognition: a review, International Journal of Agricultural Sciences, 2024.

[5] A Multitask Learning-Based Vision Transformer for Plant Disease Localization and Classification, International Journal of Computational Intelligence Systems, Vol. 17, Article 188, 2024.

Appendix

Listing 1: 1

```
import os
import random
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import Dataset, DataLoader
import torchvision.transforms as T
import torchvision.models as models
from PIL import Image

# Constants
TRAIN_IMG_DIR =
    "/root/data/Problem_B_Data/AgriculturalDisease_trainingset/images"
VAL_IMG_DIR =
    "/root/data/Problem_B_Data/AgriculturalDisease_validationset/images"
NUM_CLASSES = 61
IMG_SIZE    = 380
BATCH_SIZE  = 64
EPOCHS      = 50
LR          = 1e-3
WEIGHT_DECAY = 1e-4
DEVICE      = "cuda" if torch.cuda.is_available() else "cpu"
SEED        = 2025

# Set random seed for reproducibility
def set_seed(seed: int = SEED):
    random.seed(seed)
    torch.manual_seed(seed)
    torch.cuda.manual_seed_all(seed)

# Dataset class
class LeafDataset(Dataset):
    def __init__(self, samples, transform=None):
        self.samples = samples
        self.transform = transform
```

```
    def __len__(self):
        return len(self.samples)

    def __getitem__(self, idx):
        img_path, label = self.samples[idx]
        img = Image.open(img_path).convert("RGB")
        if self.transform:
            img = self.transform(img)
        return img, label

# Model building function
def build_model(num_classes: int):
    model =
        models.efficientnet_b4(weights=models.EfficientNet_B4_Weights.IMAGENET1K_V1)
    in_features = model.classifier[1].in_features
    model.classifier[1] = nn.Linear(in_features, num_classes)
    return model

# Training function
def train_one_epoch(model, loader, criterion, optimizer, epoch, device,
    scaler):
    model.train()
    running_loss = 0.0
    correct = 0
    total = 0

    for i, (inputs, targets) in enumerate(loader):
        inputs = inputs.to(device)
        targets = targets.to(device)

        optimizer.zero_grad()
        with torch.cuda.amp.autocast():
            outputs = model(inputs)
            loss = criterion(outputs, targets)

        scaler.scale(loss).backward()
        scaler.step(optimizer)
        scaler.update()
```

```
        running_loss += loss.item() * inputs.size(0)
        _, predicted = outputs.max(1)
        total += targets.size(0)
        correct += predicted.eq(targets).sum().item()

    return running_loss / total, correct / total

# Main training loop
def main():
    set_seed(SEED)
    print("Using device:", DEVICE)

    train_samples = list_images_with_labels(TRAIN_IMG_DIR)
    val_samples = list_images_with_labels(VAL_IMG_DIR)

    train_transform = T.Compose([
        T.RandomResizedCrop(IMG_SIZE),
        T.RandomHorizontalFlip(),
        T.ToTensor(),
        T.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225]),
    ])

    val_transform = T.Compose([
        T.Resize(int(IMG_SIZE * 1.14)),
        T.CenterCrop(IMG_SIZE),
        T.ToTensor(),
        T.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225]),
    ])

    train_dataset = LeafDataset(train_samples, transform=train_transform)
    val_dataset = LeafDataset(val_samples, transform=val_transform)

    train_loader = DataLoader(train_dataset, batch_size=BATCH_SIZE,
        shuffle=True)
    val_loader = DataLoader(val_dataset, batch_size=BATCH_SIZE, shuffle=False)

    model = build_model(NUM_CLASSES).to(DEVICE)

    criterion = nn.CrossEntropyLoss()
```

```
    optimizer = optim.AdamW(model.parameters(), lr=LR,
        weight_decay=WEIGHT_DECAY)
    scheduler = optim.lr_scheduler.CosineAnnealingLR(optimizer, T_max=EPOCHS)
    scaler = torch.cuda.amp.GradScaler()

    for epoch in range(1, EPOCHS + 1):
        train_loss, train_acc = train_one_epoch(model, train_loader, criterion,
            optimizer, epoch, DEVICE, scaler)
        print(f"Epoch [{epoch}/{EPOCHS}] Train Loss: {train_loss:.4f} Acc:
            {train_acc:.4f}")

        scheduler.step()

if __name__ == "__main__":
    main()
```

Listing 2: 2

```
import os
import random
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import Dataset, DataLoader
import torchvision.transforms as T
import torchvision.models as models

# Constants
IMG_DIR = "/path/to/training/images"
VAL_DIR = "/path/to/validation/images"
NUM_CLASSES = 61
IMG_SIZE = 224
BATCH_SIZE = 64
EPOCHS = 40
LR = 1e-3
DEVICE = "cuda" if torch.cuda.is_available() else "cpu"

# Helper function to set seed
def set_seed(seed=2025):
    random.seed(seed)
```

```
    torch.manual_seed(seed)
    torch.cuda.manual_seed_all(seed)

# Dataset class
class LeafDataset(Dataset):
    def __init__(self, samples, transform=None):
        self.samples = samples
        self.transform = transform

    def __len__(self):
        return len(self.samples)

    def __getitem__(self, idx):
        path, label = self.samples[idx]
        img = Image.open(path).convert("RGB")
        if self.transform:
            img = self.transform(img)
        return img, label

# Model building function
def build_model(num_classes):
    model = models.efficientnet_b0(pretrained=True)
    for param in model.parameters():
        param.requires_grad = False
    model.classifier[1] = nn.Linear(model.classifier[1].in_features,
        num_classes)
    return model

# Training function
def train_one_epoch(model, loader, criterion, optimizer, device):
    model.train()
    running_loss = 0.0
    correct = 0
    total = 0
    for inputs, targets in loader:
        inputs, targets = inputs.to(device), targets.to(device)
        optimizer.zero_grad()
        outputs = model(inputs)
        loss = criterion(outputs, targets)
```

```
        loss.backward()
        optimizer.step()
        running_loss += loss.item() * inputs.size(0)
        _, predicted = outputs.max(1)
        correct += predicted.eq(targets).sum().item()
        total += targets.size(0)
    return running_loss / total, correct / total

# Validation function
def validate(model, loader, criterion, device):
    model.eval()
    correct = 0
    total = 0
    with torch.no_grad():
        for inputs, targets in loader:
            inputs, targets = inputs.to(device), targets.to(device)
            outputs = model(inputs)
            _, predicted = outputs.max(1)
            correct += predicted.eq(targets).sum().item()
            total += targets.size(0)
    return correct / total

# Main loop
def main():
    set_seed()
    model = build_model(NUM_CLASSES).to(DEVICE)
    criterion = nn.CrossEntropyLoss()
    optimizer = optim.AdamW(model.parameters(), lr=LR)

    # Simulated dataset and dataloaders
    train_samples = [(f"image_{i}.jpg", i % NUM_CLASSES) for i in range(1000)]
    val_samples = [(f"val_image_{i}.jpg", i % NUM_CLASSES) for i in range(200)]

    train_loader = DataLoader(train_samples, batch_size=BATCH_SIZE)
    val_loader = DataLoader(val_samples, batch_size=BATCH_SIZE)

    for epoch in range(EPOCHS):
        train_loss, train_acc = train_one_epoch(model, train_loader, criterion,
            optimizer, DEVICE)
```

```
        val_acc = validate(model, val_loader, criterion, DEVICE)
        print(f"Epoch {epoch+1}: Train Acc: {train_acc:.4f} | Val Acc:
            {val_acc:.4f}")

if __name__ == "__main__":
    main()
```

Listing 3: 3

```
 import os
import random
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import Dataset, DataLoader
import torchvision.transforms as T
import torchvision.models as models
from torchvision.utils import save_image

# Constants
TRAIN_IMG_DIR = "/path/to/training/images"
VAL_IMG_DIR = "/path/to/validation/images"
NUM_CLASSES = 61
IMG_SIZE = 256
BATCH_SIZE = 32
EPOCHS = 25
LR = 1e-3
DEVICE = "cuda" if torch.cuda.is_available() else "cpu"
SEED = 2025
IMAGENET_MEAN = [0.485, 0.456, 0.406]
IMAGENET_STD = [0.229, 0.224, 0.225]

# Helper function to set seed
def set_seed(seed=2025):
    random.seed(seed)
    torch.manual_seed(seed)
    torch.cuda.manual_seed_all(seed)

# Dataset class
```

```
class SeverityDataset(Dataset):
    def __init__(self, paths, transform):
        self.paths = paths
        self.transform = transform

    def __len__(self):
        return len(self.paths)

    def __getitem__(self, idx):
        img_path = self.paths[idx]
        img = Image.open(img_path).convert("RGB")
        fname = os.path.basename(img_path)
        cls_id = int(fname.split("_")[0])
        sev = id_to_severity(cls_id)
        if self.transform:
            img = self.transform(img)
        return img, sev

def build_model():
    model =
        models.efficientnet_b0(weights=models.EfficientNet_B0_Weights.IMAGENET1K_V1)
    in_f = model.classifier[1].in_features
    model.classifier[1] = nn.Linear(in_f, 4)
    return model

def train_one_epoch(model, loader, criterion, optimizer, scaler):
    model.train()
    total, correct, loss_sum = 0, 0, 0
    for img, label in loader:
        img, label = img.to(DEVICE), label.to(DEVICE)
        optimizer.zero_grad()
        with torch.cuda.amp.autocast():
            out = model(img)
            loss = criterion(out, label)
        scaler.scale(loss).backward()
        scaler.step(optimizer)
        scaler.update()
        loss_sum += loss.item() * img.size(0)
        pred = out.argmax(1)
```

```
        total += img.size(0)
        correct += pred.eq(label).sum().item()
    return loss_sum / total, correct / total

def validate(model, loader, criterion):
    model.eval()
    total, correct, loss_sum = 0, 0, 0
    y_true, y_pred = [], []
    with torch.no_grad(), torch.cuda.amp.autocast():
        for img, label in loader:
            img, label = img.to(DEVICE), label.to(DEVICE)
            out = model(img)
            loss = criterion(out, label)
            loss_sum += loss.item() * img.size(0)
            pred = out.argmax(1)
            total += img.size(0)
            correct += pred.eq(label).sum().item()
            y_true.extend(label.cpu().numpy())
            y_pred.extend(pred.cpu().numpy())
    return loss_sum / total, correct / total, y_true, y_pred

def plot_training_curves(tr_l, va_l, tr_a, va_a):
    plt.figure(figsize=(12,5))
    plt.subplot(1,2,1); plt.plot(tr_l); plt.plot(va_l); plt.title("Loss")
    plt.subplot(1,2,2); plt.plot(tr_a); plt.plot(va_a); plt.title("Accuracy")
    plt.savefig("t3_training_curves.png", dpi=300); plt.close()

def plot_confusion(y_true, y_pred):
    cm = confusion_matrix(y_true, y_pred, labels=[0,1,2,3])
    plt.figure(figsize=(7,6))
    sns.heatmap(cm, annot=True, fmt="d",
                xticklabels=["Healthy","Mild","Moderate","Severe"],
                yticklabels=["Healthy","Mild","Moderate","Severe"])
    plt.savefig("t3_confusion_matrix.png", dpi=300)
    plt.close()

def main():
    set_seed(SEED)
    print("Using:", DEVICE)
```

```
# Load training and validation images
train_imgs = [
    os.path.join(TRAIN_IMG_DIR, f)
    for f in os.listdir(TRAIN_IMG_DIR)
    if is_valid_image_name(f)
]
val_imgs = [
    os.path.join(VAL_IMG_DIR, f)
    for f in os.listdir(VAL_IMG_DIR)
    if is_valid_image_name(f)
]

print(f"Train: {len(train_imgs)} images")
print(f"Val: {len(val_imgs)} images")

train_tf = T.Compose([T.RandomResizedCrop(IMG_SIZE),
    T.RandomHorizontalFlip(), T.ToTensor()])
val_tf = T.Compose([T.Resize(int(IMG_SIZE * 1.14)),
    T.CenterCrop(IMG_SIZE), T.ToTensor()])

train_ds = SeverityDataset(train_imgs, train_tf)
val_ds = SeverityDataset(val_imgs, val_tf)

train_loader = DataLoader(train_ds, batch_size=32, shuffle=True)
val_loader = DataLoader(val_ds, batch_size=32)

model = build_model().to(DEVICE)
criterion = nn.CrossEntropyLoss()
optimizer = optim.AdamW(model.parameters(), lr=1e-3)

tr_l, va_l, tr_a, va_a = [], [], [], []
for ep in range(1, 26):
    tl, ta = train_one_epoch(model, train_loader, criterion, optimizer,
        None)
    vl, va, y_true, y_pred = validate(model, val_loader, criterion)
    tr_l.append(tl); va_l.append(vl); tr_a.append(ta); va_a.append(va)

    print(f"[{ep}/25] Train Acc={ta:.4f} | Val Acc={va:.4f}")
```

```
    plot_training_curves(tr_l, va_l, tr_a, va_a)
    plot_confusion(y_true, y_pred)

    print(" Task3 Complete!")

if __name__ == "__main__":
    main()
```

Listing 4: 4

```
 import os
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import Dataset, DataLoader
import torchvision.transforms as T
import torchvision.models as models
from sklearn.metrics import confusion_matrix
import matplotlib.pyplot as plt
import seaborn as sns

# Constants
TRAIN_IMG_DIR = "/path/to/training/images"
VAL_IMG_DIR = "/path/to/validation/images"
IMG_SIZE = 256
BATCH_SIZE = 32
EPOCHS = 25
LR = 1e-3
DEVICE = "cuda" if torch.cuda.is_available() else "cpu"
NUM_CLASSES_TYPE = 61
NUM_CLASSES_SEV = 4
IMAGENET_MEAN = [0.485, 0.456, 0.406]
IMAGENET_STD = [0.229, 0.224, 0.225]

# Helper functions
def set_seed(seed=2025):
    torch.manual_seed(seed)
    torch.cuda.manual_seed_all(seed)
```

```
def is_valid_filename(fname):
    return fname.lower().endswith((".jpg", ".jpeg", ".png")) and "_" in fname
        and len(fname.split("_")[0]) <= 20

def parse_label_from_name(fname):
    return int(fname.split("_")[0])

def id_to_severity(cls):
    return 0 if cls in {0, 6, 9} else 1 if cls in {1, 3, 4} else 2 if cls ==
        16 else 3

# Dataset
class MultiTaskDataset(Dataset):
    def __init__(self, img_paths, transform):
        self.paths = img_paths
        self.transform = transform

    def __len__(self):
        return len(self.paths)

    def __getitem__(self, idx):
        path = self.paths[idx]
        img = Image.open(path).convert("RGB")
        fname = os.path.basename(path)
        cls = parse_label_from_name(fname)
        sev = id_to_severity(cls)

        if self.transform:
            img = self.transform(img)

        return img, cls, sev

# Multi-task model (EfficientNet-B4)
class EffNetB4MultiTask(nn.Module):
    def __init__(self, num_class_type, num_class_sev):
        super().__init__()
        base =
            models.efficientnet_b4(weights=models.EfficientNet_B4_Weights.IMAGENET1K_V1)
```

```
        self.features = base.features
        self.pool = base.avgpool
        self.dropout = base.classifier[0]
        feat_dim = base.classifier[1].in_features
        self.head_type = nn.Linear(feat_dim, num_class_type)
        self.head_sev = nn.Linear(feat_dim, num_class_sev)

    def forward(self, x):
        x = self.features(x)
        x = self.pool(x)
        x = torch.flatten(x, 1)
        x = self.dropout(x)
        return self.head_type(x), self.head_sev(x)

# Train function
def train_one_epoch(model, loader, criteria, optimizer, scaler):
    model.train()
    ce_type, ce_sev = criteria
    tot, correct_t, correct_s, loss_sum = 0, 0, 0, 0
    for img, cls, sev in loader:
        img, cls, sev = img.to(DEVICE), cls.to(DEVICE), sev.to(DEVICE)
        optimizer.zero_grad()
        with torch.cuda.amp.autocast():
            out_t, out_s = model(img)
            loss = ce_type(out_t, cls) + ce_sev(out_s, sev)
        scaler.scale(loss).backward()
        scaler.step(optimizer)
        scaler.update()
        loss_sum += loss.item() * img.size(0)
        pred_t, pred_s = out_t.argmax(1), out_s.argmax(1)
        tot += img.size(0)
        correct_t += pred_t.eq(cls).sum().item()
        correct_s += pred_s.eq(sev).sum().item()
    return loss_sum / tot, correct_t / tot, correct_s / tot

# Validate function
def validate(model, loader, criteria):
    model.eval()
    ce_type, ce_sev = criteria
```

```
    tot, correct_t, correct_s, loss_sum = 0, 0, 0, 0
    yt, yp, ys, yp2 = [], [], [], []
    with torch.no_grad():
        for img, cls, sev in loader:
            img, cls, sev = img.to(DEVICE), cls.to(DEVICE), sev.to(DEVICE)
            out_t, out_s = model(img)
            loss = ce_type(out_t, cls) + ce_sev(out_s, sev)
            loss_sum += loss.item() * img.size(0)
            pred_t, pred_s = out_t.argmax(1), out_s.argmax(1)
            tot += img.size(0)
            correct_t += pred_t.eq(cls).sum().item()
            correct_s += pred_s.eq(sev).sum().item()
            yt.extend(cls.cpu().numpy())
            yp.extend(pred_t.cpu().numpy())
            ys.extend(sev.cpu().numpy())
            yp2.extend(pred_s.cpu().numpy())
    return loss_sum / tot, correct_t / tot, correct_s / tot, yt, yp, ys, yp2

# Plot functions
def plot_confusion(y_true, y_pred, labels, title, save_name):
    cm = confusion_matrix(y_true, y_pred, labels=labels)
    plt.figure(figsize=(7,6))
    sns.heatmap(cm, annot=False, cmap="Blues")
    plt.title(title)
    plt.savefig(save_name, dpi=300)
    plt.close()

def plot_curve(train, val, title, save_name):
    plt.figure()
    plt.plot(train, label="Train")
    plt.plot(val, label="Val")
    plt.title(title)
    plt.legend()
    plt.savefig(save_name, dpi=300)
    plt.close()

# Main function
def main():
    set_seed(2025)
```

```
train_imgs = [os.path.join(TRAIN_IMG_DIR, f) for f in
    os.listdir(TRAIN_IMG_DIR) if is_valid_filename(f)]
val_imgs = [os.path.join(VAL_IMG_DIR, f) for f in os.listdir(VAL_IMG_DIR)
    if is_valid_filename(f)]
print(f"Train: {len(train_imgs)} images | Val: {len(val_imgs)} images")

train_tf = T.Compose([T.RandomResizedCrop(IMG_SIZE),
    T.RandomHorizontalFlip(), T.ToTensor(), T.Normalize(IMAGENET_MEAN,
    IMAGENET_STD)])
val_tf = T.Compose([T.Resize(int(IMG_SIZE * 1.14)),
    T.CenterCrop(IMG_SIZE), T.ToTensor(), T.Normalize(IMAGENET_MEAN,
    IMAGENET_STD)])

train_ds = MultiTaskDataset(train_imgs, train_tf)
val_ds = MultiTaskDataset(val_imgs, val_tf)
train_loader = DataLoader(train_ds, batch_size=BATCH_SIZE, shuffle=True,
    num_workers=4)
val_loader = DataLoader(val_ds, batch_size=BATCH_SIZE, shuffle=False,
    num_workers=4)

model = EffNetB4MultiTask(NUM_CLASSES_TYPE, NUM_CLASSES_SEV).to(DEVICE)
ce_type = nn.CrossEntropyLoss()
ce_sev = nn.CrossEntropyLoss()
criteria = (ce_type, ce_sev)
optimizer = optim.AdamW(model.parameters(), lr=LR)
scheduler = CosineAnnealingLR(optimizer, T_max=EPOCHS)
scaler = torch.cuda.amp.GradScaler()

tr_t_list, va_t_list, tr_s_list, va_s_list = [], [], [], []
for ep in range(1, EPOCHS + 1):
    tl, ta_t, ta_s = train_one_epoch(model, train_loader, criteria,
        optimizer, scaler)
    vl, va_t, va_s, yt, yp, ys, yp2 = validate(model, val_loader, criteria)
    scheduler.step()

    tr_t_list.append(ta_t)
    va_t_list.append(va_t)
    tr_s_list.append(ta_s)
    va_s_list.append(va_s)
```

```
        print(f"[{ep}/{EPOCHS}] TypeAcc={va_t:.4f} SevAcc={va_s:.4f}")

    plot_curve(tr_t_list, va_t_list, "Disease Type Accuracy",
        "type_accuracy.png")
    plot_curve(tr_s_list, va_s_list, "Severity Accuracy",
        "severity_accuracy.png")
    plot_confusion(yt, yp, list(range(NUM_CLASSES_TYPE)), "Type Confusion
        Matrix", "type_confusion.png")
    plot_confusion(ys, yp2, [0,1,2,3], "Severity Confusion Matrix",
        "severity_confusion.png")

    print("Task 4 completed!")

if __name__ == "__main__":
    main()
```